# Comparaison entre un ABR et `std::map`

Implémentation manuelle, opérations équivalentes et mesures

C++ · 200 000 valeurs distinctes · trois répétitions · rapport Typst

**Résumé**

Ce travail compare un arbre binaire de recherche (ABR) construit manuellement avec le conteneur standard `std::map<int, int>`. Les deux structures reçoivent la même permutation reproductible de 200 000 entiers distincts. Trois opérations sont mesurées séparément : insertion, recherche puis suppression. Les nouvelles mesures figurent à la fin du rapport ; les résultats varient selon l'opération. Le résultat principal est surtout algorithmique : l'ABR personnel n'offre pas de garantie logarithmique dans le pire cas, contrairement à `std::map`.

## 1. Introduction

Un arbre binaire de recherche organise ses valeurs autour d'une règle simple : les valeurs plus petites qu'un nœud vont dans le sous-arbre gauche et les valeurs plus grandes dans le sous-arbre droit. Cette propriété permet de suivre un seul chemin lors d'une recherche.

Le C++ fournit déjà une structure ordonnée prête à l'emploi : `std::map`. Elle associe une clé à une valeur et maintient les clés dans l'ordre. L'objectif est de comparer, sur les mêmes données et avec les mêmes opérations :

- l'ABR dont les nœuds, les pointeurs et la mémoire sont gérés dans le fichier `bst.cpp` ;
- `std::map<int, int>`, utilisé comme référence de la bibliothèque standard.

Le mot « sans bibliothèque » concerne l'ABR personnel : aucun arbre fourni par une bibliothèque n'est utilisé pour son implémentation. `std::map` est explicitement le second objet de comparaison.

## 2. Présentation des deux structures

### 2.1. ABR personnel

Un nœud contient une valeur et deux pointeurs. L'insertion, la recherche et la suppression parcourent l'arbre par comparaisons. Les doublons sont refusés.

```
struct Node {
    int value;
    Node* left;
    Node* right;
};
```

L'insertion est itérative :

```
Node* current = root_;
while (true) {
    if (value == current->value)
        return false;
    if (value < current->value) {
        if (current->left == nullptr) {
            current->left = new Node(value);
            return true;
        }
        current = current->left;
    } else {
        if (current->right == nullptr) {
            current->right = new Node(value);
            return true;
```

```
        }
        current = current->right;
    }
}
```

La suppression traite les trois cas classiques : feuille, un seul fils, ou deux fils. Avec deux fils, la valeur est remplacée par le successeur, c'est-à- dire le plus petit nœud du sous-arbre droit, puis ce successeur est supprimé.

### 2.2. `std::map`

Dans cette expérience, la clé et la valeur sont identiques :

```
std::map<int, int> dictionary;
dictionary.emplace(value, value);
bool present = dictionary.find(value) != dictionary.end();
dictionary.erase(value);
```

La bibliothèque prend en charge les pointeurs, les allocations et la réorganisation interne. Le standard garantit une complexité logarithmique pour la recherche, l'insertion et la suppression d'un élément.

## 3. Équivalence fonctionnelle

Les deux structures sont initialisées avec la séquence :

`50, 30, 70, 20, 40, 60, 80`

Le parcours infixe de l'ABR donne `20, 30, 40, 50, 60, 70, 80`, exactement comme l'itération sur les clés du `std::map`. Les recherches de 40 réussissent et celles de 45 échouent dans les deux structures. Après suppression de 30, les valeurs restantes sont `20, 40, 50, 60, 70, 80`.

Les tests exécutés dans le programme vérifient aussi :

- l'arbre vide et la première insertion ;
- les fils gauche et droit ;
- le rejet d'un doublon ;
- une suppression de feuille ;
- une suppression avec un seul fils ;
- une suppression avec deux fils ;
- le parcours infixe trié.

Sortie de validation :

```
OK arbre vide
OK racine
OK fils gauche et droit
OK feuilles
OK doublon ignore
OK recherche
OK parcours infixe
OK suppressions feuille et deux fils
OK valeurs supprimees
OK taille apres suppressions
OK hauteur croissante
OK comparaisons croissantes
OK suppression avec un fils
```

## 4. Méthode expérimentale

Le banc de test suit exactement le protocole suivant :

1. créer les entiers distincts `1..200_000` ;
2. les mélanger avec `std::mt19937` et la graine `2026` ;
3. mesurer l'insertion de toutes les valeurs ;
4. rechercher successivement toutes les valeurs ;

5. supprimer toutes les valeurs dans le même ordre.

Chaque structure est reconstruite pour chaque répétition. Les valeurs sont générées avant le chronométrage. Le temps d'une opération comprend donc les appels à la structure et ses allocations internes, mais pas la génération ni le mélange des données. Le chronomètre est `std::chrono::steady_clock`.

Trois répétitions sont réalisées et la moyenne est affichée en microsecondes. Le compteur de contrôle vaut `1_200_000` pour chaque structure :

> `3` répétitions × (`200 000` recherches réussies + `200 000` suppressions réussies) `= 1 200 000`

Ce contrôle confirme que les deux structures ont retrouvé et supprimé tous les éléments attendus.

## 5. Code du banc de test

La boucle appliquée à l'ABR est équivalente à celle appliquée à `std::map` :

```
for (int value : values)
    abr.insert(value);
for (int value : values)
    control += abr.contains(value);
for (int value : values)
    control += abr.erase(value);
```

Pour la référence standard :

```
for (int value : values)
    dictionary.emplace(value, value);
for (int value : values)
    control += dictionary.find(value) != dictionary.end();
for (int value : values)
    control += dictionary.erase(value);
```

## 6. Résultats précédents de performance

Machine de la mesure précédente : Intel Core i7-8665U à 1,90 GHz, 8 processeurs logiques, Ubuntu Linux x86_64. Compilation : `g++ -O2 -Wall -Wextra -pedantic`. La commande finale a été fixée au processeur logique 0 avec `taskset -c 0`.

Les valeurs sont les moyennes de trois exécutions indépendantes. Les temps sont en microsecondes (µs).

| Opération | ABR personnel | std::map | Plus rapide |
|---|---|---|---|
| Insertion de 200 000 | 49 235,983 | 51 347,795 | ABR |
| Recherche de 200 000 | 23 182,618 | 48 669,277 | ABR |
| Suppression de 200 000 | 36 118,331 | 56 963,308 | ABR |

Dans cette mesure, l'ABR est environ 1,04 fois plus rapide que `std::map` pour l'insertion, 2,10 fois plus rapide pour la recherche et 1,58 fois plus rapide pour la suppression. Ces ratios ne sont pas universels : ils dépendent du compilateur, de la machine, de l'allocateur et de la charge du système.

La hauteur observée de l'ABR après l'insertion est 42. L'ordre mélangé produit donc ici un arbre raisonnablement court. Cela explique pourquoi l'ABR ne subit pas le cas dégénéré pendant cette expérience.

## 7. Complexité théorique

Soit `h` la hauteur de l'ABR.

| Opération | ABR personnel | `std::map` |
|---|---|---|
| Insertion | O(h), moyenne O(log n), pire O(n) | O(log n) |
| Recherche | O(h), moyenne O(log n), pire O(n) | O(log n) |
| Suppression | O(h), moyenne O(log n), pire O(n) | O(log n) |
| Parcours complet | O(n) | O(n) |
| Mémoire | O(n) | O(n) |

L'ABR personnel n'est pas automatiquement équilibré. Avec une suite croissante, il devient une chaîne de hauteur `n` et le coût total de `n` insertions devient $O(n^2)$. Avec des données mélangées, sa hauteur est souvent de l'ordre de `log n`, mais cette propriété n'est pas garantie.

Le standard garantit les opérations logarithmiques de `std::map`. La structure interne concrète est masquée, mais elle est généralement fondée sur un arbre équilibré. Cette garantie est la différence algorithmique essentielle entre les deux solutions.

## 8. Avantages et limites

| ABR personnel | `std::map` |
|---|---|
| Montre directement les nœuds, les pointeurs et les parcours. | Interface standard et réutilisable. |
| Permet d'adapter la structure à un besoin pédagogique ou particulier. | Pas de gestion manuelle de la mémoire. |
| Risque de dégénérescence si l'ordre d'insertion est défavorable. | Complexité logarithmique garantie pour les opérations principales. |
| Code et maintenance à la charge du programmeur. | Structure interne et détails d'équilibrage masqués. |

Le résultat expérimental ne signifie pas que l'ABR personnel est toujours meilleur ou toujours moins bon. Il compare une implémentation pédagogique à un conteneur général, sur une machine et un ordre précis. En pratique, `std::map` est généralement le choix sûr lorsqu'une structure ordonnée fiable est nécessaire. L'ABR reste utile pour comprendre l'algorithme et pour les cas où un comportement ou un parcours personnalisé est recherché.

## 9. Conclusion

Les deux structures réalisent correctement l'insertion, la recherche et la suppression des mêmes clés. Les résultats précédents correspondent à une autre exécution ; les nouvelles mesures sont détaillées à la fin du rapport. Ce résultat de temps ne supprime toutefois pas l'avantage théorique de `std::map` : sa complexité logarithmique est garantie, alors que celle de l'ABR dépend de sa forme.

La conclusion la plus importante concerne la garantie de complexité. L'ABR personnel dépend de sa hauteur et peut devenir linéaire. `std::map` garantit une complexité logarithmique pour les opérations principales. L'ABR explique donc le fonctionnement interne d'une structure ordonnée, alors que `std::map` offre une solution standard et robuste pour un programme C++ réel.

Source et mesures : `03-arbre-binaire-recherche/bst.cpp` · graine 2026

## 10. Nouvelles mesures locales

Mesures effectuées à nouveau sur Intel Core i7-8665U à 1,90 GHz, 4 cœurs / 8 processeurs logiques, 14 Gio de RAM, Ubuntu Linux x86_64 (noyau 7.0.0-38-generic). Compilateur g++ 15.2.0, options -std=c++17 -O2 -Wall -Wextra -pedantic. Aucune affinité processeur imposée. Les valeurs ci-dessous remplacent les anciens résultats pour cette exécution et correspondent à benchmark-nouveau.csv.

Moyennes de trois exécutions sur 200 000 clés, graine 2026, en microsecondes. Le contrôle vaut 1 200 000 pour chaque structure.

| **Opération** | **ABR** | **std.map** |
|---|---|---|
| Insertion | 112715.02 | 96133.59 |
| Recherche | 51058.05 | 100791.95 |
| Suppression | 81576.88 | 103489.85 |

L’ordre d’insertion modifie fortement la hauteur. La mesure suivante utilise 5 000 clés, trois répétitions et la graine 2026 ; le cas presque trié effectue 50 échanges. Les recherches puis suppressions suivent l’ordre d’insertion. Contrôle : 30 000 par cas. Temps en microsecondes.

| **Ordre** | **Hauteur** | **Insertion** | **Recherche** | **Suppression** |
|---|---|---|---|---|
| Croissant | 5000 | 35638.5 | 45382.1 | 173.0 |
| Décroissant | 5000 | 62306.4 | 50787.4 | 247.0 |
| Aléatoire | 29 | 782.4 | 353.7 | 718.4 |
| Médianes | 13 | 496.6 | 88.4 | 316.6 |
| Presque trié | 916 | 3539.2 | 5030.8 | 368.1 |

Dans les cas triés, la suppression dans l’ordre d’insertion retire successivement la racine : ses temps ne représentent pas toutes les séquences possibles. Les arbres non équilibrés ont hauteur 5 000 ; l’ordre médian donne une hauteur de 13.

# 11. Annexe : code source complet

## 11.1. ABR et comparaison avec la bibliothèque standard

```
#include <algorithm>
#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <iomanip>
#include <iostream>
#include <map>
#include <random>
#include <string>
#include <type_traits>
#include <vector>

class BinarySearchTree {
    struct Node {
        int value;
        Node* left;
        Node* right;

        explicit Node(int value) : value(value), left(nullptr), right(nullptr) {}
    };

    Node* root_ = nullptr;
    std::size_t size_ = 0;
    std::uint64_t comparisons_ = 0;
    std::size_t height_ = 0;

public:
    BinarySearchTree() = default;

    ~BinarySearchTree() {
        clear();
    }

    BinarySearchTree(const BinarySearchTree&) = delete;
    BinarySearchTree& operator=(const BinarySearchTree&) = delete;

    bool insert(int value) {
        if (root_ == nullptr) {
            root_ = new Node(value);
            size_ = 1;
            height_ = 1;
            return true;
        }

        Node* current = root_;
        std::size_t depth = 1;
        while (true) {
            comparisons_++;
            if (value == current->value)
                return false;

            if (value < current->value) {
                if (current->left == nullptr) {
                    current->left = new Node(value);
                    size_++;
                    height_ = height_ > depth + 1 ? height_ : depth + 1;
                    return true;
                }
                current = current->left;
            } else {
                if (current->right == nullptr) {
                    current->right = new Node(value);
                    size_++;
                    height_ = height_ > depth + 1 ? height_ : depth + 1;
                    return true;
                }
```

```
            current = current->right;
        }
        depth++;
    }
}

bool contains(int value) const {
    const Node* current = root_;
    while (current != nullptr) {
        if (value == current->value)
            return true;
        current = value < current->value ? current->left : current->right;
    }
    return false;
}

bool erase(int value) {
    Node** link = &root_;
    while (*link != nullptr) {
        Node* current = *link;
        comparisons_++;
        if (value == current->value) {
            if (current->left == nullptr) {
                *link = current->right;
                delete current;
                size_--;
                return true;
            }
            if (current->right == nullptr) {
                *link = current->left;
                delete current;
                size_--;
                return true;
            }

            Node** successor_link = &current->right;
            while ((*successor_link)->left != nullptr)
                successor_link = &(*successor_link)->left;
            Node* successor = *successor_link;
            current->value = successor->value;
            *successor_link = successor->right;
            delete successor;
            size_--;
            return true;
        }
        link = value < current->value ? &current->left : &current->right;
    }
    return false;
}

void inorder(std::vector<int>& values) const {
    std::vector<const Node*> stack;
    const Node* current = root_;
    while (current != nullptr || !stack.empty()) {
        while (current != nullptr) {
            stack.push_back(current);
            current = current->left;
        }
        current = stack.back();
        stack.pop_back();
        values.push_back(current->value);
        current = current->right;
    }
}

void clear() {
    if (root_ == nullptr)
        return;

    std::vector<Node*> stack;
```

```
        stack.reserve(size_);
        stack.push_back(root_);
        while (!stack.empty()) {
            Node* current = stack.back();
            stack.pop_back();
            if (current->left != nullptr)
                stack.push_back(current->left);
            if (current->right != nullptr)
                stack.push_back(current->right);
            delete current;
        }
        root_ = nullptr;
        size_ = 0;
        comparisons_ = 0;
        height_ = 0;
    }

    std::size_t size() const {
        return size_;
    }

    std::uint64_t comparisons() const {
        return comparisons_;
    }

    std::size_t height() const {
        return height_;
    }
};

std::vector<int> make_values(std::size_t count, std::uint32_t seed) {
    std::vector<int> values(count);
    for (std::size_t i = 0; i < count; i++)
        values[i] = static_cast<int>(i + 1);
    std::mt19937 generator(seed);
    std::shuffle(values.begin(), values.end(), generator);
    return values;
}

struct OperationTimes {
    double insert_us = 0;
    double search_us = 0;
    double erase_us = 0;
    std::uint64_t control = 0;
    std::size_t height = 0;
};

template <typename Container>
OperationTimes benchmark_container(const std::vector<int>& values) {
    Container container;
    OperationTimes result;

    auto start = std::chrono::steady_clock::now();
    for (int value : values) {
        if constexpr (std::is_same_v<Container, BinarySearchTree>)
            container.insert(value);
        else
            container.emplace(value, value);
    }
    auto finish = std::chrono::steady_clock::now();
    result.insert_us = std::chrono::duration<double, std::micro>(finish - start).count();

    result.height = [&]() {
        if constexpr (std::is_same_v<Container, BinarySearchTree>)
            return container.height();
        return static_cast<std::size_t>(0);
    }();

    start = std::chrono::steady_clock::now();
    for (int value : values) {
```

```
        if constexpr (std::is_same_v<Container, BinarySearchTree>)
            result.control += container.contains(value);
        else
            result.control += container.find(value) != container.end();
    }
    finish = std::chrono::steady_clock::now();
    result.search_us = std::chrono::duration<double, std::micro>(finish - start).count();

    start = std::chrono::steady_clock::now();
    for (int value : values) {
        if constexpr (std::is_same_v<Container, BinarySearchTree>)
            result.control += container.erase(value);
        else
            result.control += container.erase(value);
    }
    finish = std::chrono::steady_clock::now();
    result.erase_us = std::chrono::duration<double, std::micro>(finish - start).count();

    return result;
}

struct Average {
    double insert_us = 0;
    double search_us = 0;
    double erase_us = 0;
    std::uint64_t control = 0;
    std::size_t height = 0;
};

template <typename Container>
Average benchmark_average(std::size_t count, std::uint32_t seed, std::size_t repetitions) {
    Average average;
    for (std::size_t repetition = 0; repetition < repetitions; repetition++) {
        std::vector<int> values = make_values(count, seed);
        OperationTimes result = benchmark_container<Container>(values);
        average.insert_us += result.insert_us;
        average.search_us += result.search_us;
        average.erase_us += result.erase_us;
        average.control += result.control;
        average.height = result.height;
    }
    average.insert_us /= repetitions;
    average.search_us /= repetitions;
    average.erase_us /= repetitions;
    return average;
}

void print_csv_row(const std::string& structure, const std::string& operation,
                   std::size_t count, std::uint32_t seed, std::size_t repetitions,
                   double microseconds, std::uint64_t control, std::size_t height) {
    std::cout << structure << ',' << operation << ',' << count << ',' << seed << ','
              << repetitions << ',' << std::fixed << std::setprecision(3)
              << microseconds << ',' << control << ',' << height << '\n';
}

bool check(bool condition, const std::string& name) {
    std::cout << (condition ? "OK " : "FAIL ") << name << '\n';
    return condition;
}

int run_tests() {
    BinarySearchTree tree;
    bool passed = true;
    passed &= check(tree.size() == 0 && tree.height() == 0, "arbre vide");
    passed &= check(tree.insert(50), "racine");
    passed &= check(tree.insert(30) && tree.insert(70), "fils gauche et droit");
    passed &= check(tree.insert(20) && tree.insert(40) && tree.insert(60) && tree.insert(80), "feuilles");
    passed &= check(!tree.insert(50), "doublon ignore");
    passed &= check(tree.contains(40) && !tree.contains(45), "recherche");
```

```
    std::vector<int> ordered;
    tree.inorder(ordered);
    passed &= check(ordered == std::vector<int>({20, 30, 40, 50, 60, 70, 80}), "parcours infixe");
    passed &= check(tree.erase(20) && tree.erase(30) && tree.erase(50), "suppressions feuille et deux
fils");
    passed &= check(!tree.contains(20) && !tree.contains(30) && !tree.contains(50), "valeurs supprimees");
    passed &= check(tree.size() == 4, "taille apres suppressions");

    BinarySearchTree chain;
    for (int value = 1; value <= 1000; value++)
        chain.insert(value);
    passed &= check(chain.height() == 1000, "hauteur croissante");
    passed &= check(chain.comparisons() == 499500, "comparaisons croissantes");
    passed &= check(chain.erase(500), "suppression avec un fils");

    return passed ? 0 : 1;
}

std::size_t parse_size(const char* text) {
    char* end = nullptr;
    unsigned long long value = std::strtoull(text, &end, 10);
    if (*text == '\0' || *end != '\0' || value == 0)
        throw std::string("valeur invalide: ") + text;
    return static_cast<std::size_t>(value);
}

std::uint32_t parse_seed(const char* text) {
    char* end = nullptr;
    unsigned long value = std::strtoul(text, &end, 10);
    if (*text == '\0' || *end != '\0' || value == 0)
        throw std::string("graine invalide: ") + text;
    return static_cast<std::uint32_t>(value);
}

int main(int argc, char** argv) {
    try {
        if (argc == 2 && std::string(argv[1]) == "--test")
            return run_tests();

        std::size_t count = 200000;
        std::size_t repetitions = 3;
        std::uint32_t seed = 2026;
        bool csv = false;

        for (int i = 1; i < argc; i++) {
            std::string argument = argv[i];
            if (argument == "--n" && i + 1 < argc)
                count = parse_size(argv[++i]);
            else if (argument == "--seed" && i + 1 < argc)
                seed = parse_seed(argv[++i]);
            else if (argument == "--runs" && i + 1 < argc)
                repetitions = parse_size(argv[++i]);
            else if (argument == "--csv")
                csv = true;
            else
                throw std::string("argument inconnu: ") + argument;
        }

        Average abr = benchmark_average<BinarySearchTree>(count, seed, repetitions);
        Average map = benchmark_average<std::map<int, int>>(count, seed, repetitions);
        if (abr.control != repetitions * count * 2 || map.control != repetitions * count * 2)
            throw std::string("echec du controle fonctionnel");

        if (csv) {
            std::cout << "structure,operation,n,seed,runs,average_us,control,height\n";
            print_csv_row("abr", "insert", count, seed, repetitions, abr.insert_us, abr.control,
abr.height);
            print_csv_row("abr", "search", count, seed, repetitions, abr.search_us, abr.control,
abr.height);
            print_csv_row("abr", "erase", count, seed, repetitions, abr.erase_us, abr.control, abr.height);
```

```
            print_csv_row("std_map", "insert", count, seed, repetitions, map.insert_us, map.control,
map.height);
            print_csv_row("std_map", "search", count, seed, repetitions, map.search_us, map.control,
map.height);
            print_csv_row("std_map", "erase", count, seed, repetitions, map.erase_us, map.control,
map.height);
        } else {
            std::cout << "n: " << count << '\n'
                      << "graine: " << seed << '\n'
                      << "repetitions: " << repetitions << '\n'
                      << "controle_abr: " << abr.control << '\n'
                      << "controle_map: " << map.control << '\n'
                      << "abr_insert_us: " << std::fixed << std::setprecision(3) << abr.insert_us << '\n'
                      << "abr_search_us: " << abr.search_us << '\n'
                      << "abr_erase_us: " << abr.erase_us << '\n'
                      << "abr_height: " << abr.height << '\n'
                      << "map_insert_us: " << map.insert_us << '\n'
                      << "map_search_us: " << map.search_us << '\n'
                      << "map_erase_us: " << map.erase_us << '\n';
        }
    } catch (const std::string& error) {
        std::cerr << error << '\n';
        return 2;
    }
    return 0;
}
```

## 11.2. Effet de l’ordre d’insertion

```
#define main bst_original_main
#include "bst.cpp"
#undef main
#include <numeric>

void balanced(std::vector<int>& result, int low, int high) {
    if (low > high) return;
    int middle = low + (high - low) / 2;
    result.push_back(middle);
    balanced(result, low, middle - 1);
    balanced(result, middle + 1, high);
}
int main() {
    constexpr int n = 5000;
    std::vector<int> ascending(n); std::iota(ascending.begin(), ascending.end(), 1);
    auto descending = ascending; std::reverse(descending.begin(), descending.end());
    auto random = make_values(n, 2026);
    std::vector<int> ideal; balanced(ideal, 1, n);
    auto nearly = ascending; std::mt19937 gen(2026); std::uniform_int_distribution<int> dist(0, n-1);
    for (int i=0;i<n/100;++i) std::swap(nearly[dist(gen)], nearly[dist(gen)]);
    std::cout << "order,n,runs,height,insert_us,search_us,erase_us,control\n";
    for (const auto& item : std::vector<std::pair<std::string,std::vector<int>>>{{"ascending",ascending},
{"descending",descending},{"random",random},{"balanced",ideal},{"almost_sorted",nearly}}) {
        double ins=0, sea=0, del=0; std::uint64_t check=0; std::size_t height=0;
        for (int run=0;run<3;++run) { auto m=benchmark_container<BinarySearchTree>(item.second);
ins+=m.insert_us; sea+=m.search_us; del+=m.erase_us; check+=m.control; height=m.height; }
        std::cout << item.first << ',' << n << ",3," << height << ',' << ins/3 << ',' << sea/3 << ',' <<
del/3 << ',' << check << '\n';
        if (check != 30000) return 1;
    }
}
```